

EnergyForecast

Manual de Usuario

Predicción de Consumo Eléctrico con XGBoost · LSTM · LLM

Proyecto Final · Inteligencia Artificial · EAFIT 2026-1

Sofia Velez · Paulina Velasquez · Alexander Vargas

Dataset: PJM Hourly
Energy

MAPE objetivo: < 5%

Stack: XGBoost + LSTM + Groq

Demo:
Streamlit

DESCRIPCIÓN DEL SISTEMA

1. ¿Qué es EnergyForecast?

EnergyForecast es un sistema de predicción horaria de demanda eléctrica entrenado sobre el dataset público **PJM Hourly Energy Consumption** (Kaggle). Combina tres modelos de complejidad creciente — ARIMA (baseline), XGBoost y LSTM — con un componente de explicabilidad basado en LLM que convierte cada predicción en lenguaje natural comprensible para el operador.

Resultados principales

Modelo	RMSE (MW)	MAPE (%)	MAE (MW)	Mejora vs baseline
Baseline ARIMA	3,121	6.5%	2,440	—
XGBoost	2,034	4.2%	1,589	−35% RMSE
LSTM (mejor)	1,842	3.8%	1,401	−41% RMSE ✓

El LSTM logra MAPE 3.8%, superando el objetivo de < 5%.

Arquitectura del sistema

■ EDA + Visualización

Análisis exploratorio de la serie temporal, descomposición estacional, ACF/PACF y detección de anomalías.

■■ Feature Engineering

Lag features (24h, 168h), rolling stats, encodings cíclicos (hora, día, mes) para capturar estacionalidad.

■ XGBoost

Modelo de gradient boosting tabular. Entrenado con walk-forward validation. Interpretado con SHAP TreeExplainer.

■ LSTM (PyTorch)

Red recurrente 2×64 unidades entrenada en Google Colab (GPU T4). Captura dependencias a largo plazo.

■ LLM — LLaMA 3.1 (Groq)

Few-shot + chain-of-thought sobre los valores SHAP top-5. Genera explicación en español para el operador.

■■ Demo Streamlit

Interfaz web con 3 pestañas: predicción interactiva + explicación LLM, métricas comparativas y análisis SHAP global.

ANTES DE EMPEZAR

2. Requisitos del Sistema

Asegúrate de cumplir los siguientes requisitos antes de instalar el sistema.

Software requerido

Componente	Versión mínima	Notas
Python	3.9+	Recomendado 3.10 o 3.11
pip	22+	Incluido con Python
Git	2.x	Para clonar el repositorio
Navegador	Chrome / Firefox	Para usar la demo Streamlit

Hardware recomendado

Componente	Mínimo	Recomendado
RAM	4 GB	8 GB o más
CPU	Dual-core	Quad-core o más
GPU	No requerida*	NVIDIA CUDA para re-entrenar LSTM
Disco	2 GB libres	5 GB para modelos y datos

* El LSTM fue entrenado en Google Colab (GPU T4 gratuita). Para re-entrenarlo localmente se recomienda GPU. La demo Streamlit y el modelo XGBoost corren perfectamente sin GPU.

Cuenta de Groq (para explicaciones LLM)

✓ GROQ ES GRATUITO Y NO REQUIERE TARJETA

El componente de explicación con LLM usa la **API de Groq** (tier gratuito, sin tarjeta de crédito). Sin la API Key, el sistema funciona igual pero sin las explicaciones en lenguaje natural. Ver Sección 3 — paso 5 para obtenerla.

PUESTA EN MARCHA

3. Instalación Paso a Paso

Sigue estos pasos en orden. El proceso completo toma aproximadamente 5–10 minutos en una conexión normal.

1

Clonar el repositorio

Abre una terminal y ejecuta el siguiente comando:

● ● ●

bash

```
git clone https://github.com/equipo-ia-eafit/energy-forecast-2026.git cd
energy-forecast-2026
```

2

Crear entorno virtual (recomendado)

Aísla las dependencias del proyecto del resto de tu sistema:

● ● ●

bash

```
python -m venv venv # Windows: venv\Scripts\activate # macOS/Linux: source
venv/bin/activate
```

3

Instalar dependencias

Instala todas las librerías necesarias con un solo comando:

● ● ●

bash

```
pip install -r requirements.txt
```

4

Descargar el dataset

El dataset no está incluido en el repositorio por su tamaño. Descárgalo manualmente:

● ● ●

bash

```
1. Ve a: kaggle.com/datasets/robikscube/hourly-energy-consumption 2. Descarga el
archivo PJME_hourly.csv 3. Colócalo en la carpeta: data/raw/PJME_hourly.csv
```

5

Obtener API Key de Groq (gratuita)

Para activar las explicaciones en lenguaje natural:

● ● ●

bash

```
1. Regístrate en: console.groq.com 2. Ve a API Keys → Create Key 3. Copia la key
(empieza con gsk_...) 4. Guárdala – la necesitarás en el paso 7
```

6

Ejecutar los notebooks en orden

Los notebooks generan los modelos entrenados y los archivos procesados que necesita la demo:



bash

```
01_eda.ipynb → EDA y visualizaciones 02_preprocessing.ipynb → Features y secuencias LSTM
03_modeling.ipynb → Entrena ARIMA, XGBoost y LSTM 04_llm_rag_agents.ipynb → Integración con Groq LLM
```

7

Lanzar la demo Streamlit

Con los modelos entrenados, inicia la interfaz web:



bash

```
# Configura la API Key primero: export GROQ_API_KEY=gsk_tu_key_aqui # macOS/Linux
set GROQ_API_KEY=gsk_tu_key_aqui # Windows # Luego lanza la app: streamlit run main.py
```

■■ ERROR MÁS COMÚN AL INICIAR

Si aparece el error **FileNotFoundError: models/checkpoints/xgb_energy.pkl**, significa que aún no has ejecutado el notebook 03_modeling.ipynb. Ejecútalo completo y vuelve a lanzar la demo.

4. Cómo Usar la Demo Streamlit

Al ejecutar **streamlit run main.py**, se abre automáticamente el navegador en **http://localhost:8501**. La interfaz tiene un panel lateral y tres pestañas principales.

Panel lateral (Sidebar)

Elemento del sidebar	Descripción
Logo EAFIT + nombre del sistema	Identificación del proyecto
Campo 'Groq API Key' (tipo contraseña)	Ingresa tu key gsk_... aquí. Aparece  Key configurada al guardar
Lista del equipo	Integrantes del proyecto
Pie: Universidad EAFIT · 2026-1	Información académica

Pestaña 1 — Predicción + Explicación

Es la pestaña principal y más interactiva del sistema. Permite seleccionar cualquier fecha y hora del set de prueba (2016–2018) y obtener una predicción instantánea junto con su explicación en lenguaje natural.

Paso	Control	Descripción
1	Selector de fecha	Elige cualquier día del test set (2016–2018). El sistema solo permite fechas con datos reales.
2	Slider de hora (0–23)	Selecciona la hora del día. El sistema busca automáticamente el índice más cercano en los datos.
3	Botón  Predecir y Explicar	Ejecuta la predicción con XGBoost y calcula los valores SHAP. Activa también la llamada al LLM si hay API Key.
4	Métricas (3 columnas)	Muestra: Predicción (MW) Real (MW) Error (%). Permite comparar el modelo con el valor real.
5	Factores SHAP (top 5)	Tarjetas con los 5 features más influyentes. Rojo ▲ = aumentan el consumo, Verde ▼ = lo reducen.
6	Explicación LLM	Texto en español generado por LLaMA 3.1 vía Groq. Explica la predicción en 4 oraciones usando los SHAP values. Requiere API Key.

SIN API KEY: LA PREDICCIÓN IGUAL FUNCIONA

Si no ingresas la API Key de Groq, el sistema igual muestra la predicción y los factores SHAP. Solo la sección 'Explicación LLM' estará deshabilitada con un aviso en amarillo.

Pestaña 2 — Métricas del modelo

Muestra la comparación objetiva entre los tres modelos del sistema y una gráfica de predicciones vs. valores reales sobre la primera semana del test set.

Elemento	Descripción
Tabla de resultados	RMSE, MAPE y MAE de los tres modelos. Las celdas mínimas se resaltan en verde.
Texto de benchmark	Resumen automático: 'El LSTM supera al baseline en 41% en RMSE'.
Gráfico de barras (×3)	Un barplot por métrica (RMSE, MAPE, MAE) con los valores de cada modelo.
Serie temporal	Predicción XGBoost vs. real sobre los primeros 7 días del test (168 horas).

Pestaña 3 — Análisis SHAP

Muestra la importancia global de los features según los valores SHAP calculados sobre una muestra de 500 observaciones del test set. Es útil para entender qué variables son más determinantes en el consumo eléctrico.

Feature	Interpretación
lag_24h / lag_168h	El consumo de las últimas 24h y de la misma hora la semana pasada son los predictores más fuertes.
hour_sin / hour_cos	El patrón cíclico diario es fundamental — hay picos de consumo en la mañana y en la tarde.
roll_mean_24h	La media móvil de 24h captura la 'inercia' del sistema eléctrico.
month_sin / month_cos	La estacionalidad anual: más consumo en verano (aire acondicionado) e invierno (calefacción).
dayofweek_*	Días laborales vs. fin de semana tienen patrones claramente distintos.

PIPELINE TÉCNICO

5. Guía de los Notebooks

Los notebooks deben ejecutarse en orden. Cada uno genera archivos que el siguiente necesita como entrada.

■ 01_eda.ipynb

EDA y Visualizaciones — Carga el dataset crudo PJME_hourly.csv y realiza el análisis exploratorio completo.

- Carga y limpieza de datos: parsing de fechas, detección de valores nulos
- Visualización de la serie temporal completa (2002–2018)
- Descomposición estacional (tendencia, estacionalidad, residuos)
- Análisis de distribución, outliers y ciclos diarios/semanales/anuales
- ACF y PACF para identificar estructura de autocorrelación

■ Genera: `data/processed/fig01–fig05` (figuras para el informe)

■ 02_preprocessing.ipynb

Feature Engineering — Transforma la serie temporal en features tabulares para XGBoost y secuencias para LSTM.

- Lag features: `lag_1h`, `lag_24h`, `lag_48h`, `lag_168h` (misma hora semana pasada)
- Rolling statistics: media y desviación estándar en ventanas de 24h y 168h
- Encodings cíclicos: `hour_sin/cos`, `dayofweek_sin/cos`, `month_sin/cos`
- Walk-forward split: train hasta 2015, val 2016, test 2016–2018
- Creación de secuencias LSTM: ventana de 168 horas (1 semana)

■ Genera: `data/processed/X_test_xgb.csv`, `y_test_xgb.csv`, `feature_cols.json`

■ 03_modeling.ipynb

Entrenamiento y Evaluación — Entrena los tres modelos y reporta las métricas comparativas.

- ARIMA (baseline): orden (2,1,2) determinado por ACF/PACF
- XGBoost: 500 árboles, `max_depth=6`, `learning_rate=0.05`. Walk-forward CV
- LSTM: 2 capas × 64 unidades, `dropout=0.2`. Entrenado 30 épocas en Colab
- SHAP TreeExplainer sobre XGBoost: importancia local y global
- Guarda: `models/checkpoints/xgb_energy.pkl`

■ Genera: `xgb_energy.pkl` (modelo serializado con joblib)

■ 04_llm_rag_agents.ipynb

Integración LLM — Conecta el sistema con LLaMA 3.1 vía Groq y valida la calidad de las explicaciones.

- Configuración del cliente Groq con `GROQ_API_KEY`
- Diseño del prompt: system + few-shot con 2 ejemplos + chain-of-thought
- Test de latencia: promedio < 2 segundos por explicación
- Evaluación cualitativa sobre 10 casos del test set
- Demostración del loop completo: predicción → SHAP → LLM → texto

■ Requiere: `GROQ_API_KEY` configurada como variable de entorno

PREGUNTAS FRECUENTES

6. FAQ

P: ¿Qué hago si la demo muestra el error 'No se encontraron los archivos del modelo'?

R: Asegúrate de haber ejecutado el notebook 03_modeling.ipynb completo. Este notebook guarda el archivo models/checkpoints/xgb_energy.pkl que la demo necesita. Luego vuelve a ejecutar: `streamlit run main.py`

P: ¿La API Key de Groq tiene costo?

R: No. Groq ofrece un tier gratuito suficiente para el uso normal de la demo. Ve a console.groq.com → Sign Up → API Keys → Create Key. No requiere tarjeta de crédito.

P: ¿Puedo usar el sistema sin la API Key de Groq?

R: Sí. Sin la API Key, la predicción XGBoost y el análisis SHAP funcionan normalmente. Solo la pestaña de 'Explicación LLM' mostrará un aviso en amarillo y no generará texto.

P: ¿Por qué el selector de fecha está limitado a 2016–2018?

R: Ese rango corresponde al test set, es decir, datos que el modelo nunca vio durante el entrenamiento. Usarlo garantiza que las métricas reflejan la capacidad real de generalización del modelo.

P: ¿Cómo re-entreno el LSTM en mi computador sin GPU?

R: El LSTM original se entrenó en Google Colab con GPU T4 gratuita (30 épocas ≈ 25 min). Sin GPU, el entrenamiento tarda aproximadamente 3–4 horas en CPU. Puedes reducir las épocas a 10 para una prueba rápida cambiando `NUM_EPOCHS = 10` en el notebook 03.

P: ¿En qué unidades están las predicciones?

R: Todas las predicciones están en MW (megavatios). El dataset PJM cubre la región PJM Interconnection en el este de Estados Unidos. El consumo típico oscila entre 20,000 y 60,000 MW dependiendo de la hora y la estación.

P: ¿Qué significa el MAPE de 3.8%?

R: MAPE (Mean Absolute Percentage Error) indica el error promedio como porcentaje del valor real. Un MAPE de 3.8% significa que, en promedio, el modelo se equivoca en $\pm 3.8\%$ del consumo real. Para el contexto de predicción eléctrica, un $\text{MAPE} < 5\%$ es considerado muy bueno en la literatura.

P: El servidor de Streamlit dice 'Address already in use'. ¿Qué hago?

R: El puerto 8501 está ocupado por otra instancia de Streamlit. Cierra la sesión anterior o usa un puerto distinto: `streamlit run main.py --server.port 8502`

REFERENCIA RÁPIDA

7. Comandos y Rutas Clave

Comandos esenciales

```
bash

# Activar entorno virtual source venv/bin/activate # macOS/Linux
venv\Scripts\activate # Windows # Instalar dependencias pip install -r
requirements.txt # Configurar API Key export GROQ_API_KEY=gsk_... # macOS/Linux set
GROQ_API_KEY=gsk_... # Windows # Lanzar la demo streamlit run main.py # Lanzar en un
puerto diferente streamlit run main.py --server.port 8502 # Ejecutar notebooks (con
Jupyter) jupyter notebook
```

Estructura de archivos clave

Ruta	Descripción
main.py	Demo Streamlit — punto de entrada de la interfaz
requirements.txt	Dependencias del proyecto
data/raw/PJME_hourly.csv	Dataset original (debes descargarlo de Kaggle)
data/processed/X_test_xgb.csv	Features del test set para XGBoost (generado por notebook 02)
data/processed/y_test_xgb.csv	Valores reales del test set (generado por notebook 02)
data/processed/feature_cols.json	Lista de features usados por el modelo (generado por notebook 02)
models/checkpoints/xgb_energy.pkl	Modelo XGBoost serializado (generado por notebook 03)
notebooks/01_eda.ipynb	EDA y análisis exploratorio
notebooks/02_preprocessing.ipynb	Feature engineering y preparación de datos
notebooks/03_modeling.ipynb	Entrenamiento y evaluación de modelos
notebooks/04_llm_rag_agents.ipynb	Integración con LLM vía Groq

Variables de entorno

Variable	Valor	Descripción
GROQ_API_KEY	gsk_...	API Key de Groq para activar explicaciones LLM

■ RECURSOS DEL PROYECTO

El repositorio completo está disponible en GitHub. El video demo (≤ 3 minutos) muestra el sistema funcionando en tiempo real. El informe técnico completo está en docs/informe_final.pdf (generado con LaTeX en Overleaf).